LAPORAN PRAKTIKUM KEAMANAN SIBER
PERTEMUAN 06
Implementasi Enkripsi dan Dekripsi Simetris dengan AES-256-GCM



Penulis

Nuevalen Refitra Alswando 103072430008

Tanggal Praktikum

25 Oktober 2026

PROGRAM STUDI INFORMATIKA

DIREKTORAT KAMPUS SURABAYA

UNIVERSITAS TELKOM

2026/2027

Daftar Isi

BAB I PENDAHULUAN 3
1.1 Tujuan Praktikum 3
1.2 Latar Belakang 3
BAB II DASAR TEORI 3
2.1 Prinsip Kerckhoffs 3
2.2 AES-256-GCM dan konsep (AEAD) 4
2.3 Key Derivation Function (PBKDF2) 4
2.4 Nonce (Number Used Once) 5
BAB III IMPLEMENTASI PROGRAM 5
3.1 Struktur Program 5
3.2 Diagram Alur Enkripsi 5
3.3 Alur Proses Dekripsi 6
3.4 Penanganan Kesalahan 6
3.5 Kode Program Lengkap 6
BAB IV HASIL PENGUJIAN 8
4.1 Skenario 1 - Teks Biasa 8
4.2 Skenario 2 - Unicode & Simbol 9
4.3 Skenario 3 - Kata Sandi Salah 9
4.4 Skenario 4 - Ciphertext Rusak 10
4.5 Ringkasan Hasil Pengujian 10
BAB V ANALISIS PERAN KUNCI, SALT, DAN NONCE 11
5.1 Analisis Peran Kata Sandi dan Kunci 11
5.2 Analisis Peran Salt (16 byte) 11
5.3 Analisis Peran Nonce (12 byte) 12
5.4 Analisis Ciphertext Berbeda Setiap Enkripsi 12
BAB VI JAWABAN PERTANYAAN REFLEKSI 13
6.1 Pertanyaan 1 13
6.2 Pertanyaan 2 13
6.3 Pertanyaan 3 14
BAB VII KESIMPULAN 14
BAB VIII DAFTAR PUSTAKA 15

BAB I
PENDAHULUAN

1.1 Tujuan Praktikum

Praktikum ini diselenggarakan dengan beberapa tujuan utama yang saling berkaitan. Pertama, saya bertujuan untuk memahami konsep fundamental dalam kriptografi simetris, khususnya mengenai plaintext, ciphertext, dan kunci rahasia. Kedua, saya perlu mengimplementasikan dua fitur utama yaitu enkripsi teks dan dekripsi teks menggunakan algoritma AES-256-GCM dalam bentuk aplikasi berbasis command line interface. Ketiga, saya harus menguji kemampuan sistem dalam memulihkan plaintext serta menangani kasus ketika kunci yang dimasukkan salah atau ciphertext mengalami kerusakan.

1.2 Latar Belakang

Berdasarkan materi mingu 06 yang telah dipelajari, kriptografi simetris merupakan fondasi utama dalam perlindungan kerahasiaan data, baik saat data sedang ditransmisikan melalui jaringan maupun saat data disimpan dalam media penyimpanan. Standar modern yang direkomendasikan dalam industri keamanan siber saat ini adalah AES-256-GCM karena algoritma ini menyediakan tiga jaminan keamanan sekaligus dalam satu kesatuan yang disebut AEAD atau Authenticated Encryption with Associated Data. Ketiga jaminan tersebut meliputi kerahasiaan data untuk memastikan informasi tidak dapat dibaca oleh pihak yang tidak berwenang, integritas data untuk memastikan informasi tidak mengalami perubahan selama perjalanan, dan keaslian sumber data untuk memastikan informasi benar-benar berasal dari pengirim yang sah.

Praktikum ini mengimplementasikan standar tersebut dalam bentuk aplikasi berbasis terminal menggunakan bahasa pemrograman Python. Pemilihan Python sebagai bahasa implementasi didasarkan pada kemudahan penggunaan dan ketersediaan library kriptografi yang komprehensif, khususnya library cryptography yang menyediakan implementasi AES-GCM yang telah teruji dan aman untuk digunakan.

BAB II
DASAR TEORI

2.1 Prinsip Kerckhoffs

"Keamanan sistem kriptografi harus bergantung pada kerahasiaan KUNCI, bukan pada kerahasiaan algoritma." — Auguste Kerckhoffs (1883)

Dalam mempelajari kriptografi, saya memahami bahwa fondasi utama desain sistem kriptografi modern adalah Prinsip Kerckhoffs yang dirumuskan oleh Auguste Kerckhoffs pada tahun 1883. Prinsip ini menyatakan bahwa keamanan sistem kriptografi harus bergantung sepenuhnya pada kerahasiaan kunci, bukan pada kerahasiaan algoritma atau rancangannya. Artinya, algoritma enkripsi boleh dan bahkan sebaiknya dipublikasikan secara terbuka agar dapat dievaluasi, diuji coba, dan diserang secara independen oleh pakar kriptografi di seluruh

dunia. Keamanan sistem murni bertumpu pada kekuatan matematika algoritma dan pengelolaan kunci rahasia yang baik.

Prinsip ini menjadi sangat penting karena membedakan pendekatan keamanan modern dari pendekatan security by obscurity yang mengandalkan anggapan bahwa penyerang tidak mengetahui cara kerja internal algoritma. Pendekatan security by obscurity sangat rapuh karena ketika cara kerja sistem terbongkar melalui proses reverse engineering, seluruh keamanan akan runtuh seketika. Sebaliknya, dengan Prinsip Kerckhoffs, kita selalu berasumsi bahwa penyerang mengetahui sistem secara detail, tetapi tidak memiliki kuncinya.

2.2 AES-256-GCM dan konsep (AEAD)

Advanced Encryption Standard atau AES merupakan standar enkripsi modern dunia yang ditetapkan oleh NIST dalam FIPS 197 pada tanggal 26 November 2001 dengan pembaruan resmi terakhir pada 9 Mei 2023. AES berbasis algoritma Rijndael yang dikembangkan oleh Joan Daemen dan Vincent Rijmen. Algoritma ini menggunakan arsitektur Substitution-Permutation Network atau SPN, bukan struktur Feistel seperti yang digunakan pada DES. Seluruh 128 bit blok diproses secara simultan pada setiap ronde melalui empat transformasi utama yaitu SubBytes, ShiftRows, MixColumns, dan AddRoundKey.

AES memiliki ukuran blok tetap 128 bit dengan tiga pilihan panjang kunci yaitu 128 bit, 192 bit, atau 256 bit. Jumlah ronde pemrosesan bervariasi tergantung panjang kunci yang digunakan, yaitu 10 ronde untuk AES-128, 12 ronde untuk AES-192, dan 14 ronde untuk AES-256. Dalam praktikum ini, saya menggunakan AES-256 yang merupakan pilihan terkuat dengan tingkat keamanan tertinggi.

Galois/Counter Mode atau GCM merupakan mode operasi block cipher yang berbasis pada Counter Mode dan memberikan authentication tag sepanjang 16 byte untuk menjamin integritas dan autentikasi data. Kombinasi AES dengan GCM menghasilkan apa yang disebut sebagai AEAD atau Authenticated Encryption with Associated Data. AEAD merupakan standar emas dalam protokol TLS 1.3 dan komunikasi internet modern karena memberikan tiga jaminan keamanan sekaligus dalam satu operasi kriptografi.

2.3 Key Derivation Function (PBKDF2)

Dalam implementasi sistem kriptografi nyata, kata sandi yang digunakan oleh manusia bersifat pendek dan lemah sehingga tidak dapat langsung digunakan sebagai kunci enkripsi. Diperlukan mekanisme untuk memperkuat kata sandi tersebut menjadi kunci kriptografi yang kuat. Mekanisme ini disebut Key Derivation Function atau KDF.

Dalam praktikum ini, saya menggunakan PBKDF2-HMAC-SHA256 dengan 200.000 iterasi dan salt acak sepanjang 16 byte. PBKDF2 atau Password-Based Key Derivation Function 2 merupakan standar yang direkomendasikan dalam RFC 2898 untuk menurunkan kunci dari kata sandi. Fungsi ini bekerja dengan cara mengulang proses hashing sebanyak 200.000 kali, yang secara signifikan memperlambat serangan brute-force. Salt acak yang digabungkan dengan kata sandi sebelum proses hashing berfungsi untuk mencegah serangan rainbow table

dan memastikan bahwa dua pengguna dengan kata sandi identik tetap menghasilkan kunci yang berbeda.

2.4 Nonce (Number Used Once)

Nonce atau Number Used Once merupakan nilai acak sepanjang 12 byte yang unik untuk setiap proses enkripsi. Dalam mode GCM yang berbasis pada Counter Mode, pengulangan nonce dengan kunci yang sama merupakan bencana keamanan total karena keystream yang dihasilkan akan identik. Jika dua plaintext berbeda dienkripsi dengan keystream yang sama, penyerang dapat melakukan operasi XOR pada kedua ciphertext untuk mendapatkan XOR dari kedua plaintext, yang dapat membocorkan informasi sensitif.

Oleh karena itu, dalam implementasi ini, saya memastikan bahwa setiap proses enkripsi membangkitkan nonce acak baru menggunakan fungsi os.urandom dari Python. Meskipun nonce tidak perlu dirahasiakan dan dapat disimpan bersama ciphertext, nonce wajib unik untuk setiap enkripsi dengan kunci yang sama.

BAB III
IMPLEMENTASI PROGRAM

3.1 Struktur Program

Setelah mempelajari teori dan spesifikasi tugas, saya mengimplementasikan program Python yang terdiri dari empat fungsi utama. Fungsi pertama adalah turunkan_kunci yang bertugas menurunkan kunci sepanjang 32 byte dari kata sandi dan salt menggunakan PBKDF2-HMAC-SHA256. Fungsi kedua adalah enkripsi yang menerima plaintext dan kata sandi sebagai input, kemudian menghasilkan ciphertext dalam format Base64. Fungsi ketiga adalah dekripsi yang menerima ciphertext Base64 dan kata sandi, kemudian memulihkan plaintext asli atau memberikan pesan kesalahan jika terjadi masalah. Fungsi keempat adalah main yang menyediakan menu interaktif berbasis command line untuk pengguna.

3.2 Diagram Alur Enkripsi

Dalam proses enkripsi, pertama-tama saya membangkitkan salt acak sepanjang 16 byte dan nonce acak sepanjang 12 byte menggunakan fungsi os.urandom. Kedua nilai acak ini memastikan bahwa setiap proses enkripsi menghasilkan output yang berbeda meskipun input plaintext dan kata sandi sama. Selanjutnya, saya menurunkan kunci AES-256 sepanjang 32 byte dari kata sandi dan salt menggunakan fungsi PBKDF2 dengan 200.000 iterasi.

Setelah kunci berhasil diturunkan, saya membuat objek AESGCM menggunakan kunci tersebut. Plaintext yang berupa string diubah menjadi bytes menggunakan encoding UTF-8, kemudian dienkripsi menggunakan metode encrypt dari objek AESGCM. Metode ini menerima tiga parameter yaitu nonce, plaintext bytes, dan associated data yang dalam kasus ini saya set sebagai None. Hasil enkripsi berupa ciphertext bytes yang sudah termasuk authentication tag sepanjang 16 byte.

Langkah terakhir dalam proses enkripsi adalah menggabungkan salt, nonce, dan ciphertext menjadi satu payload utuh, kemudian mengencode payload tersebut menggunakan Base64 agar dapat ditampilkan sebagai teks yang aman untuk ditransmisikan atau disimpa

3.3 Alur Proses Dekripsi

Proses dekripsi merupakan kebalikan dari proses enkripsi. Pertama, saya melakukan decode Base64 pada ciphertext yang diterima untuk mendapatkan payload bytes. Saya juga melakukan validasi panjang payload untuk memastikan bahwa payload minimal memiliki panjang 44 byte yang terdiri dari 16 byte salt, 12 byte nonce, dan minimal 16 byte untuk ciphertext beserta tag.

Selanjutnya, saya memisahkan payload menjadi tiga bagian yaitu salt dari byte ke-0 hingga ke-16, nonce dari byte ke-16 hingga ke-28, dan ciphertext bytes dari byte ke-28 hingga akhir. Menggunakan salt yang sama dan kata sandi yang dimasukkan pengguna, saya menurunkan kembali kunci AES-256 menggunakan PBKDF2.

Dengan kunci yang telah diturunkan dan nonce yang telah diekstrak, saya membuat objek AESGCM baru dan memanggil metode decrypt. Metode ini secara otomatis memverifikasi authentication tag yang tertanam dalam ciphertext bytes. Jika kata sandi salah atau ciphertext telah dimodifikasi, verifikasi tag akan gagal dan metode decrypt akan memunculkan exception InvalidTag. Jika verifikasi berhasil, metode decrypt mengembalikan plaintext bytes yang kemudian saya decode menjadi string menggunakan UTF-8.

3.4 Penanganan Kesalahan

Dalam implementasi ini, saya juga menambahkan mekanisme penanganan kesalahan yang komprehensif. Jika terjadi InvalidTag yang menunjukkan kata sandi salah atau ciphertext telah dimodifikasi, program akan menampilkan pesan kesalahan yang jelas tanpa menampilkan traceback mentah yang membingungkan pengguna. Saya juga menangani ValueError dan UnicodeDecodeError yang mungkin terjadi jika format ciphertext tidak valid atau terjadi masalah dalam decoding UTF-8. Semua exception lainnya juga ditangkap dan ditampilkan dengan pesan yang informatif.

3.5 Kode Program Lengkap

```
"""
Implementasi Enkripsi dan Dekripsi Simetris dengan AES-256-GCM.
Program CLI untuk praktikum Keamanan Siber - Pertemuan 06.
Nama file: NIM_Nama_AES.py
"""

import base64
import hashlib
import os

from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.exceptions import InvalidTag

def turunkan_kunci(kata_sandi: str, salt: bytes) -> bytes:
    """Menurunkan kunci AES-256 (32 byte) dari kata sandi menggunakan PBKDF2-HMAC-SHA256."""
    return hashlib.pbkdf2_hmac(
        "sha256",
        kata_sandi.encode("utf-8"),
        salt,
        iterations=200_000,
```

```
        dklen=32,
    )

def enkripsi(teks: str, kata_sandi: str) -> str:
    if not teks:
        raise ValueError("Plaintext tidak boleh kosong.")

    salt = os.urandom(16)
    nonce = os.urandom(12)

    kunci = turunkan_kunci(kata_sandi, salt)
    aesgcm = AESGCM(kunci)

    plaintext_bytes = teks.encode("utf-8")
    ciphertext_bytes = aesgcm.encrypt(nonce, plaintext_bytes, None)

    payload = salt + nonce + ciphertext_bytes
    return base64.b64encode(payload).decode("utf-8")

def dekripsi(ciphertext_b64: str, kata_sandi: str) -> str:
    try:
        payload = base64.b64decode(ciphertext_b64, validate=True)
    except (base64.binascii.Error, ValueError) as exc:
        raise ValueError(f"Format Base64 tidak valid: {exc}") from exc

    if len(payload) < 44:
        raise ValueError(
            f"Ciphertext terlalu pendek (diterima {len(payload)} byte, minimal 44 byte)."
        )

    salt = payload[:16]
    nonce = payload[16:28]
    ciphertext_bytes = payload[28:]

    kunci = turunkan_kunci(kata_sandi, salt)
    aesgcm = AESGCM(kunci)
    plaintext_bytes = aesgcm.decrypt(nonce, ciphertext_bytes, None)

    return plaintext_bytes.decode("utf-8")

def main() -> None:
    while True:
        print("\n=== ENKRIPSI SIMETRIS (AES-256-GCM) ===")
        print("1. Enkripsi teks")
        print("2. Dekripsi teks")
        print("0. Keluar")
        pilihan = input("Pilih menu: ").strip()

        if pilihan == "0":
            print("Program selesai.")
            break

        if pilihan not in {"1", "2"}:
            print("Pilihan menu tidak valid. Silakan pilih 1, 2, atau 0.")
            continue

        if pilihan == "1":
            pesan = input("Masukkan plaintext: ")
        else:
            pesan = input("Masukkan ciphertext Base64: ")

        kata_sandi = input("Masukkan kata sandi: ")
        if not kata_sandi:
            print("Kata sandi tidak boleh kosong.")
            continue
```

```
    try:
        if pilihan == "1":
            hasil = enkripsi(pesan, kata_sandi)
            print("Ciphertext (Base64):", hasil)
        else:
            hasil = dekripsi(pesan, kata_sandi)
            print("Plaintext:", hasil)
    except InvalidTag:
        print("Dekripsi gagal: kata sandi salah atau ciphertext telah dimodifikasi.")
    except ValueError as ve:
        print(f"Dekripsi gagal: {ve}")
    except UnicodeDecodeError:
        print("Dekripsi gagal: hasil dekripsi bukan teks UTF-8 yang valid.")
    except Exception as exc:
        print(f"Terjadi kesalahan tak terduga: {exc}")

if __name__ == "__main__":
    main()
```

BAB IV
HASIL PENGUJIAN

4.1 Skenario 1 - Teks Biasa

```
=== ENKRIPSI SIMETRIS (AES-256-GCM) ===
1. Enkripsi teks
2. Dekripsi teks
0. Keluar
Pilih menu: 1
Masukkan plaintext: Halo Informatika
Masukkan kata sandi: rahasia123
Ciphertext (Base64): oTbBsnKk1zuGgnPL/RsGPlxeexcbwPbw7gX5YjwKsOv+eRkB7pT0yKasE/PAYArW9J0XD+uc5AW+paG1

=== ENKRIPSI SIMETRIS (AES-256-GCM) ===
1. Enkripsi teks
2. Dekripsi teks
0. Keluar
Pilih menu: 2
Masukkan ciphertext Base64: oTbBsnKk1zuGgnPL/RsGPlxeexcbwPbw7gX5YjwKsOv+eRkB7pT0yKasE/PAYArW9J0XD+uc5AW+paG1
Masukkan kata sandi: rahasia123
Plaintext: Halo Informatika
```

Setelah program berhasil diimplementasikan, saya melakukan pengujian pertama dengan menggunakan teks biasa. Saya memasukkan plaintext "Halo Informatika" dan kata sandi "rahasia123" ke dalam menu enkripsi. Program berhasil menghasilkan ciphertext dalam format Base64 yang terdiri dari rangkaian karakter alfanumerik dengan panjang tertentu.

Selanjutnya, saya menyalin ciphertext Base64 tersebut dan memasukkannya ke dalam menu dekripsi bersama dengan kata sandi yang sama yaitu "rahasia123". Setelah proses dekripsi selesai, program berhasil memulihkan plaintext asli yaitu "Halo Informatika" yang identik dengan input awal. Hasil pengujian ini menunjukkan bahwa proses enkripsi dan dekripsi berjalan dengan benar dan plaintext dapat dipulihkan secara sempurna.

Status pengujian ini adalah LULUS karena plaintext hasil dekripsi identik dengan input awal.

4.2 Skenario 2 - Unicode & Simbol

```
=== ENKRIPSI SIMETRIS (AES-256-GCM) ===
1. Enkripsi teks
2. Dekripsi teks
0. Keluar
Pilih menu: 1
Masukkan plaintext: Kampus 🎓 & Keamanan$Siber!
Masukkan kata sandi: rahasia123
Ciphertext (Base64): VqnQx3hm54QO6KHag2W6w4BznAMy8U70greQ2Aq3Pe49bfTsS/jtCwvIfSB47OAyCh4UAbQ38g3iGPWTpS0FgbsM59MR7EJ4jg==

=== ENKRIPSI SIMETRIS (AES-256-GCM) ===
1. Enkripsi teks
2. Dekripsi teks
0. Keluar
Pilih menu: 2
Masukkan ciphertext Base64: VqnQx3hm54QO6KHag2W6w4BznAMy8U70greQ2Aq3Pe49bfTsS/jtCwvIfSB47OAyCh4UAbQ38g3iGPWTpS0FgbsM59MR7EJ4jg==
Masukkan kata sandi: rahasia123
Plaintext: Kampus 🎓 & Keamanan$Siber!
```

Untuk menguji kemampuan program dalam menangani karakter khusus, saya melakukan pengujian kedua dengan menggunakan teks yang mengandung emoji dan simbol. Saya memasukkan plaintext "Kampus 🎓 & Keamanan$Siber!" yang mengandung emoji graduation cap, simbol ampersand, dan simbol dollar. Kata sandi yang digunakan tetap "rahasia123".

Program berhasil mengenkripsi teks tersebut menjadi ciphertext Base64. Ketika saya melakukan dekripsi dengan kata sandi yang sama, program berhasil memulihkan plaintext asli "Kampus 🎓 & Keamanan$Siber!" secara sempurna tanpa ada karakter yang hilang atau berubah. Ini menunjukkan bahwa program mampu menangani encoding UTF-8 dengan benar, termasuk karakter Unicode multi-byte seperti emoji.

Status pengujian ini adalah LULUS karena teks Unicode dan simbol tetap utuh setelah proses dekripsi.

4.3 Skenario 3 - Kata Sandi Salah

```
=== ENKRIPSI SIMETRIS (AES-256-GCM) ===
1. Enkripsi teks
2. Dekripsi teks
0. Keluar
Pilih menu: 1
Masukkan plaintext: kata sandi salah
Masukkan kata sandi: rahasia123
Ciphertext (Base64): iCarBK44UYgqM+fyZeJWK40G7fagUqd/noZbx8WRn1fOI0gy1vHx1Q/GjUjcHNkHuqBQJ93YutInfjZN

=== ENKRIPSI SIMETRIS (AES-256-GCM) ===
1. Enkripsi teks
2. Dekripsi teks
0. Keluar
Pilih menu: 2
Masukkan ciphertext Base64: iCarBK44UYgqM+fyZeJWK40G7fagUqd/noZbx8WRn1fOI0gy1vHx1Q/GjUjcHNkHuqBQJ93YutInfjZN
Masukkan kata sandi: salah_banget
Dekripsi gagal: kata sandi salah atau ciphertext telah dimodifikasi.
```

Untuk menguji mekanisme keamanan program, saya melakukan pengujian ketiga dengan menggunakan kata sandi yang salah. Saya menggunakan ciphertext yang dihasilkan dari skenario pertama, tetapi memasukkan kata sandi "salah_banget" yang berbeda dari kata sandi asli "rahasia123".

Ketika proses dekripsi dijalankan, program tidak menghasilkan plaintext palsu atau data acak. Sebaliknya, program mendeteksi bahwa kata sandi tidak sesuai dan menampilkan pesan kesalahan "Dekripsi gagal: kata sandi salah atau ciphertext telah dimodifikasi." Mekanisme ini bekerja karena AES-GCM memverifikasi authentication tag yang tertanam dalam ciphertext. Ketika kata sandi salah, kunci yang diturunkan juga salah, sehingga tag yang dihitung tidak cocok dengan tag yang tersimpan dalam ciphertext, dan metode decrypt memunculkan exception InvalidTag.

Status pengujian ini adalah LULUS karena program menolak dekripsi dengan kata sandi salah tanpa menampilkan plaintext palsu, yang menunjukkan bahwa mekanisme autentikasi dan integritas bekerja dengan baik.

4.4 Skenario 4 - Ciphertext Rusak

```
=== ENKRIPSI SIMETRIS (AES-256-GCM) ===
1. Enkripsi teks
2. Dekripsi teks
0. Keluar
Pilih menu: 1
Masukkan plaintext: Halo Informatika
Masukkan kata sandi: rahasia123
Ciphertext (Base64): a/W5f5F4l+3QhNZP/Q4KkRg7zyPiYBeqsxepmW6zbEpYq3nJSZUKqykqGzZwytH+TYcs9TM+c4Q45pD/

=== ENKRIPSI SIMETRIS (AES-256-GCM) ===
1. Enkripsi teks
2. Dekripsi teks
0. Keluar
Pilih menu: 2
Masukkan ciphertext Base64: a/W5f5F4l+3QhNZP/Q4KkRg7zyPiYBeqsxepmJ6zbEpYq3nJSZUKqykqGzZwytH+TYcs9TM+c4Q45pD/
Masukkan kata sandi: rahasia123
Dekripsi gagal: kata sandi salah atau ciphertext telah dimodifikasi.
```

Pengujian keempat bertujuan untuk menguji integritas data. Saya mengambil ciphertext dari skenario pertama dan mengubah satu karakter di tengah ciphertext tersebut, kemudian mencoba mendekripsinya dengan kata sandi yang benar yaitu "rahasia123".

Meskipun kata sandi yang dimasukkan benar, program tetap menolak untuk mendekripsi ciphertext yang telah dimodifikasi tersebut. Program menampilkan pesan kesalahan "Dekripsi gagal: kata sandi salah atau ciphertext telah dimodifikasi." Hal ini terjadi karena perubahan sekecil apa pun pada ciphertext akan menyebabkan authentication tag tidak valid, dan AES-GCM akan mendeteksi modifikasi tersebut melalui verifikasi tag.

Status pengujian ini adalah LULUS karena AES-GCM berhasil mendeteksi modifikasi ciphertext dan menolak dekripsi, yang menunjukkan bahwa jaminan integritas data bekerja dengan sempurna.

4.5 Ringkasan Hasil Pengujian

Setelah melakukan keempat skenario pengujian di atas, saya dapat menyimpulkan bahwa seluruh pengujian berhasil dengan status LULUS. Skenario pertama menunjukkan bahwa enkripsi dan dekripsi teks biasa berjalan dengan benar. Skenario kedua menunjukkan kemampuan program dalam menangani karakter Unicode dan simbol. Skenario ketiga menunjukkan bahwa program dapat mendeteksi dan menolak dekripsi dengan kata sandi

salah. Skenario keempat menunjukkan bahwa program dapat mendeteksi modifikasi ciphertext dan menjaga integritas data.

BAB V
ANALISIS PERAN KUNCI, SALT, DAN NONCE

5.1 Analisis Peran Kata Sandi dan Kunci

Setelah melakukan praktikum ini, saya menganalisis bahwa kata sandi yang dimasukkan oleh pengguna tidak langsung digunakan sebagai kunci enkripsi AES. Kata sandi manusia bersifat pendek, mudah ditebak, dan tidak memiliki entropi yang cukup untuk menjadi kunci kriptografi yang kuat. Oleh karena itu, diperlukan mekanisme untuk memperkuat kata sandi tersebut.

Dalam implementasi ini, saya menggunakan fungsi PBKDF2-HMAC-SHA256 untuk menurunkan kunci sepanjang 32 byte dari kata sandi dan salt. Fungsi ini bekerja dengan cara mengulang proses hashing sebanyak 200.000 kali, yang secara signifikan memperlambat proses penurunan kunci. Meskipun ini memperlambat proses enkripsi dan dekripsi normal, hal ini juga memperlambat serangan brute-force secara proporsional. Penyerang yang mencoba menebak kata sandi harus melakukan 200.000 iterasi untuk setiap tebakan, sehingga biaya komputasi serangan meningkat drastis dibandingkan dengan fungsi hash tunggal.

Namun, perlu dicatat bahwa perlambatan ini bersifat proporsional, bukan mutlak. PBKDF2 bukanlah fungsi memory-hard, sehingga tetap dapat dieksekusi secara efisien pada GPU atau ASIC. Untuk kata sandi berentropi rendah seperti "rahasia123", serangan berbasis GPU masih layak dilakukan dalam waktu yang masuk akal. OWASP Password Storage Cheat Sheet (edisi 2023) saat ini merekomendasikan minimal 600.000 iterasi untuk PBKDF2-HMAC-SHA256, atau penggunaan KDF memory-hard seperti Argon2id atau scrypt untuk aplikasi produksi yang menghadapi ancaman perangkat keras paralel.

Dalam konteks praktikum ini, 200.000 iterasi sudah memadai untuk tujuan pembelajaran karena menunjukkan mekanisme key stretching dengan jelas, sementara kata sandi yang digunakan diasumsikan cukup kuat. Untuk implementasi nyata di luar lingkungan akademis, peningkatan jumlah iterasi atau migrasi ke Argon2id menjadi pertimbangan yang penting.

Analisis saya menunjukkan bahwa mekanisme ini sesuai dengan Prinsip Kerckhoffs karena keamanan sistem bertumpu sepenuhnya pada kerahasiaan kata sandi yang digunakan untuk menurunkan kunci. Algoritma PBKDF2 dan AES-GCM bersifat publik, tetapi tanpa mengetahui kata sandi yang benar, penyerang tidak dapat menurunkan kunci yang tepat untuk mendekripsi ciphertext.

5.2 Analisis Peran Salt (16 byte)

Dalam praktikum ini, saya menganalisis bahwa salt merupakan nilai acak sepanjang 16 byte yang digabungkan dengan kata sandi sebelum proses hashing dalam PBKDF2. Salt tidak perlu dirahasiakan dan disimpan bersama ciphertext dalam output Base64.

Fungsi utama salt adalah untuk mencegah serangan rainbow table. Tanpa salt, penyerang dapat membuat tabel hash yang sudah dihitung sebelumnya untuk kata sandi umum, dan

menggunakan tabel tersebut untuk mencari kata sandi dengan cepat. Dengan salt, setiap kata sandi menghasilkan hash yang berbeda meskipun kata sandinya sama, sehingga penyerang harus menghitung ulang hash untuk setiap salt yang berbeda.

Selain itu, salt juga memastikan bahwa dua pengguna dengan kata sandi identik tetap menghasilkan kunci enkripsi yang berbeda. Ini penting dalam skenario di mana beberapa pengguna memilih kata sandi yang sama, karena ciphertext mereka tetap akan berbeda dan tidak dapat dikorelasikan.

5.3 Analisis Peran Nonce (12 byte)

Nonce atau Number Used Once merupakan nilai acak sepanjang 12 byte yang unik untuk setiap proses enkripsi. Dalam praktikum ini, saya menganalisis bahwa nonce memiliki peran yang sangat kritis dalam keamanan AES-GCM.

AES-GCM berbasis pada Counter Mode yang menghasilkan keystream dengan mengenkripsi nilai counter. Jika nonce yang sama digunakan dua kali dengan kunci yang sama, keystream yang dihasilkan akan identik. Dalam kasus ini, jika dua plaintext P1 dan P2 dienkripsi dengan keystream K yang sama, ciphertext yang dihasilkan adalah C1 = P1 XOR K dan C2 = P2 XOR K. Penyerang dapat melakukan XOR pada kedua ciphertext untuk mendapatkan C1 XOR C2 = P1 XOR P2, yang membocorkan informasi tentang plaintext.

Oleh karena itu, nonce wajib unik untuk setiap enkripsi dengan kunci yang sama. Dalam implementasi ini, saya menggunakan os.urandom untuk membangkitkan nonce acak yang berbeda setiap kali. Meskipun nonce tidak perlu dirahasiakan, pengulangan nonce merupakan bencana keamanan yang harus dihindari.

5.4 Analisis Ciphertext Berbeda Setiap Enkripsi

Dalam pengujian, saya mengamati bahwa ketika saya mengenkripsi teks yang sama "Halo Informatika" dengan kata sandi yang sama "rahasia123" dua kali, dihasilkan ciphertext Base64 yang berbeda. Saya menganalisis bahwa hal ini terjadi karena setiap proses enkripsi membangkitkan salt acak 16 byte dan nonce acak 12 byte yang baru menggunakan os.urandom.

Kombinasi salt dan nonce yang berbeda menyebabkan kunci turunan yang berbeda dan keystream yang berbeda, sehingga ciphertext akhir pasti berbeda. Ini merupakan fitur keamanan yang diinginkan karena mencegah pattern leakage. Jika ciphertext selalu sama untuk plaintext yang sama, penyerang dapat mengetahui bahwa dua pesan identik sedang dikirim, yang dapat membocorkan informasi sensitif.

Fitur ini juga sesuai dengan prinsip semantic security dalam kriptografi modern, yang menyatakan bahwa ciphertext tidak boleh membocorkan informasi apa pun tentang plaintext, bahkan informasi bahwa dua plaintext identik sedang dikirim.

BAB VI
JAWABAN PERTANYAAN REFLEKSI

6.1 Pertanyaan 1

Mengapa kata sandi yang sama diperlukan ketika melakukan dekripsi?

Setelah melakukan praktikum ini, saya memahami bahwa kata sandi yang sama diperlukan ketika melakukan dekripsi karena kriptografi simetris menggunakan kunci yang sama untuk enkripsi dan dekripsi, sesuai dengan Prinsip Kerckhoffs. Dalam implementasi ini, kata sandi diturunkan menjadi kunci AES-256 melalui PBKDF2-HMAC-SHA256 dengan salt yang sama.

Jika kata sandi yang dimasukkan berbeda dari kata sandi yang digunakan saat enkripsi, maka kunci yang dihasilkan juga akan berbeda. Ketika kunci yang berbeda ini digunakan untuk mendekripsi ciphertext, proses dekripsi akan gagal karena AES-GCM secara otomatis mendeteksi ketidakcocokan melalui verifikasi authentication tag sepanjang 16 byte yang tertanam dalam ciphertext. Jika tag yang dihitung dengan kunci baru tidak cocok dengan tag yang tersimpan, metode decrypt akan memunculkan exception InvalidTag.

Mekanisme ini sekaligus berfungsi sebagai autentikasi implisit. Hanya pemegang kata sandi yang benar yang dapat menurunkan kunci yang tepat dan mendekripsi pesan dengan sukses. Ini merupakan keunggulan mode AEAD yang dibahas dalam materi KamSib 06 tentang Message Authentication Code, di mana enkripsi dan autentikasi dilakukan dalam satu operasi terpadu.

6.2 Pertanyaan 2

Mengapa ciphertext perlu dikonversi ke Base64 ketika ditampilkan sebagai teks?

Dalam praktikum ini, saya menganalisis bahwa ciphertext hasil AES-GCM merupakan data biner mentah yang terdiri dari byte acak dengan nilai dari 0 hingga 255. Banyak dari nilai byte ini merupakan karakter non-printable yang tidak dapat ditampilkan dengan benar di terminal atau disimpan dalam format teks biasa.

Jika ciphertext biner ini ditampilkan langsung tanpa encoding, data akan rusak atau tidak terbaca. Karakter non-printable dapat menyebabkan masalah dalam terminal, tidak dapat disalin dan ditempel dengan benar, dan dapat kehilangan integritas saat dikirim melalui saluran teks seperti email, JSON, atau URL.

Base64 merupakan skema encoding yang mengonversi byte biner menjadi teks ASCII yang aman. Base64 menggunakan 64 karakter yaitu A hingga Z, a hingga z, 0 hingga 9, serta simbol + dan /, dengan = sebagai padding. Dengan encoding ini, ciphertext biner dapat ditampilkan di terminal, disalin tanpa corrupt, dan dikirim melalui saluran teks apa pun tanpa kehilangan integritas.

Hal ini sesuai dengan praktik Encryption at Rest dan Data in Transit yang dibahas dalam materi KamSib 06, di mana data terenkripsi harus tetap dapat ditransmisikan dan disimpan tanpa kehilangan integritas. Base64 memastikan bahwa ciphertext dapat ditangani sebagai teks biasa oleh sistem dan aplikasi yang hanya mendukung teks.

6.3 Pertanyaan 3

Mengapa proses enkripsi dengan plaintext dan kata sandi yang sama dapat menghasilkan ciphertext berbeda?

Setelah melakukan pengujian dan analisis, saya memahami bahwa proses enkripsi dengan plaintext dan kata sandi yang sama dapat menghasilkan ciphertext berbeda karena setiap proses enkripsi membangkitkan salt acak sepanjang 16 byte dan nonce acak sepanjang 12 byte yang baru menggunakan fungsi os.urandom.

Kombinasi salt dan nonce yang berbeda ini menyebabkan beberapa perbedaan dalam proses enkripsi. Pertama, kunci turunan yang dihasilkan oleh PBKDF2 akan berbeda karena salt yang berbeda. Kedua, keystream yang dihasilkan oleh AES-GCM akan berbeda karena nonce yang berbeda. Ketiga, hasil XOR plaintext dengan keystream yang berbeda akan menghasilkan ciphertext akhir yang berbeda.

Ini merupakan fitur keamanan yang disengaja, bukan bug atau kesalahan implementasi. Fitur ini mencegah pattern leakage, yaitu situasi di mana penyerang dapat mengetahui bahwa dua pesan identik sedang dikirim hanya dengan mengamati ciphertext. Dengan memastikan bahwa ciphertext selalu berbeda meskipun plaintext dan kata sandi sama, sistem memenuhi prinsip semantic security yang menyatakan bahwa ciphertext tidak boleh membocorkan informasi apa pun tentang plaintext.

Fitur ini mencegah *pattern leakage* dan analisis frekuensi. Perlu dicatat bahwa keacakan nonce tidak dengan sendirinya mencegah serangan replay, pencegahannya memerlukan mekanisme tambahan seperti nomor urut pesan atau timestamp pada lapisan protokol.

Hal ini sesuai dengan materi KamSib 06 tentang Initialization Vector pada mode CBC dan Counter Mode, di mana nilai acak unik digunakan agar dua pesan dengan awalan identik menghasilkan ciphertext awal yang berbeda.

BAB VII
KESIMPULAN

Setelah melakukan praktikum ini dengan lengkap, saya dapat menyimpulkan beberapa poin penting. Pertama, praktikum ini berhasil mendemonstrasikan implementasi kriptografi simetris modern dengan standar AES-256-GCM sesuai dengan materi KamSib 06. Prinsip Kerckhoffs diterapkan dengan benar di mana algoritma AES-GCM bersifat publik, tetapi keamanan bergantung sepenuhnya pada kerahasiaan kata sandi.

Kedua, AES-256-GCM sebagai implementasi AEAD memberikan tiga jaminan keamanan sekaligus yaitu kerahasiaan, integritas, dan autentikasi. Hal ini menjadikannya standar emas dalam protokol TLS 1.3 dan komunikasi internet modern. Dalam praktikum ini, saya mengamati bagaimana AES-GCM secara otomatis menolak dekripsi dengan kata sandi salah atau ciphertext yang telah dimodifikasi melalui mekanisme verifikasi authentication tag.

Ketiga, PBKDF2-HMAC-SHA256 dengan 200.000 iterasi dan salt acak sepanjang 16 byte berhasil mengubah kata sandi manusia yang lemah menjadi kunci sepanjang 256 bit yang tahan terhadap serangan brute-force. Mekanisme ini juga mencegah serangan rainbow table dan memastikan bahwa kata sandi identik menghasilkan kunci yang berbeda.

Keempat, nonce acak sepanjang 12 byte menjamin bahwa ciphertext berbeda setiap kali enkripsi dilakukan, meskipun plaintext dan kata sandi sama. Hal ini mencegah pattern leakage dan nonce reuse attack yang dapat membocorkan plaintext.

Kelima, Base64 encoding memungkinkan ciphertext biner ditampilkan sebagai teks yang aman untuk transmisi dan penyimpanan melalui saluran teks biasa.

Keenam, penanganan error melalui exception InvalidTag bekerja dengan baik, di mana program menolak dekripsi dengan kunci salah atau ciphertext rusak tanpa membocorkan plaintext atau menampilkan traceback yang membingungkan.

Implementasi ini menjadi fondasi pemahaman saya menuju materi minggu berikutnya tentang Public-Key Cryptography dan Distribusi Kunci di Internet Global, yang merupakan solusi atas keterbatasan kriptografi simetris dalam hal distribusi kunci melalui saluran yang tidak aman.

BAB VIII
DAFTAR PUSTAKA

1. KamSib 06 - Symmetric Cryptography. Program Studi S1 Informatika, Fakultas Informatika, Universitas Telkom.
2. NIST FIPS 197 - Advanced Encryption Standard (AES). National Institute of Standards and Technology.
3. NIST SP 800-38D - Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and MAC. National Institute of Standards and Technology.
4. RFC 2898 - PKCS #5: Password-Based Cryptography Specification Version 2.0. Internet Engineering Task Force.
5. Library cryptography Python. https://cryptography.io

BAB IX

1. File Praktikum : https://github.com/ValenNz/Implementasi-Enkripsi-dan-Dekripsi-Simetris-dengan-AES-256-GCM